
Mathematics for AI

A Beginner-to-Intermediate Foundation

Linear Algebra · Probability · Statistics · Calculus

The exact set of mathematical ideas you need before diving into machine learning and deep learning — explained with intuition, worked examples, NumPy code, and practice problems.

Topics Covered

Scalars, Vectors, Matrices & Tensors · Matrix Multiplication
Eigenvalues · Probability Fundamentals · Bayes' Theorem
Mean, Median, Mode · Variance & Standard Deviation
Correlation · Normal Distribution · Gradients

A Self-Study Guide

Prepared for Sahil

How to Use This Book

This guide is built to be worked through in short, focused sessions — roughly an hour at a time. You do not need any mathematics beyond high-school algebra to start. Every chapter builds on the ones before it, so read in order the first time through.

Throughout the book you will see four kinds of highlighted boxes:

The plain-English picture behind a formula. If the algebra ever feels dry, these boxes tell you what is really going on.

A direct link from the concept to how it shows up in machine learning and deep learning. This is why you are learning the maths in the first place.

Worked Example

A fully worked numerical example. Do these by hand at least once — the understanding comes from the doing, not the reading.

The single sentence you should remember from a section. If you revise nothing else, revise these.

Code appears in NumPy, the standard numerical library for AI in Python. Type each snippet out and run it yourself rather than copying it wholesale — that habit is where the intuition sticks. Each chapter ends with practice problems; full solutions follow immediately so you can check your work.

Read a little, compute a lot.

Contents

How to Use This Book	2
I Linear Algebra	5
1 The Language of Linear Algebra	7
1.1 Scalars, Vectors, Matrices, Tensors	7
1.1.1 Scalar — a single number	7
1.1.2 Vector — an ordered list of numbers	7
1.1.3 Matrix — a grid of numbers	7
1.1.4 Tensor — any number of dimensions	8
1.2 Basic Vector Operations	8
1.2.1 The dot product	8
1.2.2 Length (norm) of a vector	9
2 Matrix Multiplication	11
2.1 The Rule	11
2.1.1 The shape rule — the most important line in this chapter	11
2.2 Matrix Times Vector	12
2.3 Properties You Must Know	12
2.3.1 Two special matrices	12
3 Eigenvalues and Eigenvectors (Concept)	14
3.1 A Matrix as a Transformation	14
3.2 The Definition	14
3.3 Why AI Cares	15
3.4 A Few Facts to Carry Forward	15
II Probability and Statistics	17
4 Probability Fundamentals	19
4.1 The Basic Vocabulary	19
4.2 Combining Events	19
4.2.1 Complement — the “not” rule	19
4.2.2 Addition — the “or” rule	19
4.2.3 Multiplication — the “and” rule	20
4.3 Conditional Probability	20
5 Bayes’ Theorem	22
5.1 Deriving It	22
5.2 Reading the Formula	22

5.3	The Classic Medical-Test Example	22
6	Mean, Median, and Mode	25
6.1	The Three Measures of Centre	25
6.1.1	Mean — the average	25
6.1.2	Median — the middle	25
6.1.3	Mode — the most common	25
6.2	Which One Should You Use?	25
7	Variance and Standard Deviation	28
7.1	Building Up the Idea	28
7.1.1	Variance	28
7.1.2	Standard deviation	28
7.2	A Note on n versus $n - 1$	29
8	Correlation	31
8.1	From Covariance to Correlation	31
8.1.1	Covariance	31
8.1.2	The correlation coefficient	31
8.2	The Two Great Warnings	32
8.2.1	Correlation is not causation	32
8.2.2	Correlation only sees straight lines	32
9	The Normal Distribution	34
9.1	Shape and Parameters	34
9.2	The 68–95–99.7 Rule	34
9.3	Standardising: the z-score	35
III	Calculus	37
10	Introduction to Calculus and Gradients	39
10.1	Slope: the Rate of Change	39
10.2	The Derivative	39
10.2.1	A few rules you will actually use	39
10.3	The Gradient: Slope in Many Directions	40
10.4	Gradient Descent: How Models Learn	41
IV	Appendices	43
A	NumPy Quick Reference	44
A.1	Creating arrays	44
A.2	Shapes and axes	44
A.3	Linear algebra	44
A.4	Statistics	44
B	Glossary	46
C	Where to Go Next	48

Part I

Linear Algebra

Linear algebra is the mathematics of *arrays of numbers* — lists, grids, and higher-dimensional stacks — and the operations that transform them. Almost every quantity inside a neural network (an input image, a set of weights, a batch of predictions) is one of these arrays. If you understand linear algebra, you understand the shape of the data flowing through every AI model.

Chapter 1

The Language of Linear Algebra

Before we can do anything useful we need names for the objects we work with. Linear algebra has exactly four, distinguished only by how many *directions* (axes) of numbers they hold.

1.1 Scalars, Vectors, Matrices, Tensors

1.1.1 Scalar — a single number

A **scalar** is just one number: a temperature of 23, a learning rate of 0.01, a probability of 0.7. It has no direction and no shape. We write scalars in lowercase italics: a , x , λ .

$$x = 5, \quad \lambda = 0.001$$

1.1.2 Vector — an ordered list of numbers

A **vector** is a one-dimensional array: a column (or row) of numbers stacked in order. We write vectors in lowercase bold, $\mathbf{v}$, and usually as columns:

$$\mathbf{v} = \begin{bmatrix} 2 \\ -1 \\ 4 \end{bmatrix}, \quad \mathbf{v} \in \mathbb{R}^3.$$

The notation $\mathbb{R}^3$ means “a vector of 3 real numbers.” Each number is a *component*. The **dimension** of a vector is simply how many components it has.

A vector is both a *list* and an *arrow*. The vector $\begin{bmatrix} 3 \\ 4 \end{bmatrix}$ can be read as the list “3 then 4” or as an arrow pointing 3 units right and 4 units up. Both readings are useful: the list view is how a computer stores it; the arrow view is how you picture distance, direction, and angle.

1.1.3 Matrix — a grid of numbers

A **matrix** is a two-dimensional array: rows and columns. We write matrices in uppercase bold, $\mathbf{A}$. A matrix with m rows and n columns is an $m \times n$ matrix (“ m by n ”, rows first, always).

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathbf{A} \in \mathbb{R}^{2 \times 3}.$$

The entry in row i , column j is written A_{ij} . Here $A_{12} = 2$ and $A_{23} = 6$.

1.1.4 Tensor — any number of dimensions

A **tensor** is the general term for an array with *any* number of axes. In this language a scalar is a 0-axis tensor, a vector is a 1-axis tensor, a matrix is a 2-axis tensor, and a stack of matrices is a 3-axis tensor — and so on. The number of axes is called the **rank** (or *order*) of the tensor.

Object	Axes (rank)	Example shape	Everyday example
Scalar	0	()	a single pixel's brightness
Vector	1	(3,)	RGB colour of one pixel
Matrix	2	(28, 28)	a grayscale image
3-tensor	3	(28, 28, 3)	a colour image (H×W×RGB)
4-tensor	4	(64, 28, 28, 3)	a <i>batch</i> of 64 colour images

“Tensor” is the T in TensorFlow, and the core object in PyTorch. When you load a batch of colour images into a deep-learning model, you hand it a rank-4 tensor of shape (batch, height, width, channels). Debugging deep-learning code is, more often than not, the art of getting tensor *shapes* to line up. Learning to read a shape like (64, 28, 28, 3) at a glance is a genuine day-one skill.

1.2 Basic Vector Operations

Two vectors of the *same* dimension can be added component by component, and any vector can be scaled by a scalar:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}, \quad 3 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

1.2.1 The dot product

The single most important vector operation in AI is the **dot product**. Multiply matching components and add up the results:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n.$$

The dot product turns two vectors into a single scalar. For $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$:

$$\mathbf{a} \cdot \mathbf{b} = (1)(4) + (2)(5) + (3)(6) = 4 + 10 + 18 = 32.$$

The dot product measures how much two vectors “agree.” It is large and positive when they point the same way, zero when they are perpendicular, and negative when they point in opposite directions. That is why it is used everywhere a model needs to score similarity — between a search query and a document, or between two word embeddings.

1.2.2 Length (norm) of a vector

The **length** or **Euclidean norm** of a vector is the straight-line distance from the origin to its tip — Pythagoras in n dimensions:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{\mathbf{v} \cdot \mathbf{v}}.$$

For $\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$, the length is $\sqrt{9 + 16} = \sqrt{25} = 5$.

Norms measure the “size” of things models care about. The size of a weight vector appears in *regularisation* (penalising large weights to prevent overfitting). The size of the difference between prediction and target is a *loss*. And normalising a vector to length 1 is the first step in cosine similarity, the backbone of modern search and recommendation systems.

Listing 1.1: Vectors in NumPy

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)          # [5 7 9]          element-wise add
print(3 * a)          # [3 6 9]          scale
print(np.dot(a, b))   # 32              dot product
print(a @ b)          # 32              '@' is the dot/matmul operator
print(np.linalg.norm(np.array([3, 4]))) # 5.0      length
```

A scalar has no axes, a vector has one, a matrix has two, and a tensor has any number. The dot product collapses two equal-length vectors into one number that measures their agreement.

Practice Problems

1. Let $\mathbf{u} = \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}$ and $\mathbf{w} = \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}$. Compute $\mathbf{u} + \mathbf{w}$, $2\mathbf{u}$, and $\mathbf{u} \cdot \mathbf{w}$.
2. Find the length of $\mathbf{w}$ above.
3. A colour video has 30 frames, each 1080×1920 pixels with 3 colour channels. What is the rank of the tensor that stores it, and what is its shape?
4. Two unit vectors are perpendicular. What is their dot product?

Solutions.

$$1. \mathbf{u} + \mathbf{w} = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix}; 2\mathbf{u} = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix}; \mathbf{u} \cdot \mathbf{w} = (2)(1) + (0)(3) + (-1)(2) = 0.$$

2. $\|\mathbf{w}\| = \sqrt{1 + 9 + 4} = \sqrt{14} \approx 3.74$.
3. Rank 4; shape (30, 1080, 1920, 3) — (frames, height, width, channels).
4. 0. A dot product of zero means the vectors are perpendicular (orthogonal). Notice $\mathbf{u}$ and $\mathbf{w}$ in problem 1 were already perpendicular.

Chapter 2

Matrix Multiplication

Matrix multiplication is the engine of AI. A single forward pass through a neural network layer *is* a matrix multiplication followed by a simple function. Get comfortable here and a huge amount of machine learning stops looking like magic.

2.1 The Rule

To multiply $\mathbf{A}$ by $\mathbf{B}$, take the dot product of each *row* of $\mathbf{A}$ with each *column* of $\mathbf{B}$. The entry in row i , column j of the result is

$$(\mathbf{AB})_{ij} = \sum_k A_{ik} B_{kj} = (\text{row } i \text{ of } \mathbf{A}) \cdot (\text{column } j \text{ of } \mathbf{B}).$$

2.1.1 The shape rule — the most important line in this chapter

You can only multiply $\mathbf{AB}$ if the *inner* dimensions match:

$$\underbrace{(m \times n)}_{\mathbf{A}} \underbrace{(n \times p)}_{\mathbf{B}} \longrightarrow (m \times p).$$

The two inner n 's must be equal; they cancel, and the result takes the *outer* dimensions $m \times p$.

Think of it as “the inside numbers must agree, the outside numbers survive.” A (2×3) times a (3×4) works and gives a (2×4) . A (2×3) times a (2×4) fails — the 3 and the 2 in the middle do not match. More than half of all deep-learning bugs are exactly this mismatch.

Worked Example

Multiply $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ by $\mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

Both are 2×2 ; inner dimensions match, so the result is 2×2 .

$$(\mathbf{AB})_{11} = (1)(5) + (2)(7) = 5 + 14 = 19$$

$$(\mathbf{AB})_{12} = (1)(6) + (2)(8) = 6 + 16 = 22$$

$$(\mathbf{AB})_{21} = (3)(5) + (4)(7) = 15 + 28 = 43$$

$$(\mathbf{AB})_{22} = (3)(6) + (4)(8) = 18 + 32 = 50$$

$$\mathbf{AB} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

2.2 Matrix Times Vector

A vector is just a matrix with one column, so the same rule applies. An $(m \times n)$ matrix times an $(n \times 1)$ vector gives an $(m \times 1)$ vector:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} (1)(1) + (2)(0) + (3)(-1) \\ (4)(1) + (5)(0) + (6)(-1) \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

One layer of a neural network computes $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$: the input vector $\mathbf{x}$ is multiplied by a weight matrix $\mathbf{W}$, then a bias vector $\mathbf{b}$ is added. That matrix–vector product *is* the layer. Stacking many such layers, each followed by a nonlinear function, is exactly what a deep network does. When people say a GPU is good at AI, they mean it is good at doing enormous matrix multiplications very fast.

2.3 Properties You Must Know

- **Not commutative:** in general $\mathbf{AB} \neq \mathbf{BA}$. Order matters. Often one order is not even a valid shape.
- **Associative:** $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.
- **Distributive:** $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$.

2.3.1 Two special matrices

The **identity matrix** $\mathbf{I}$ has 1s on the diagonal and 0s elsewhere. It is the “1” of matrices: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$.

$$\mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The **transpose** $\mathbf{A}^\top$ flips a matrix over its diagonal, turning rows into columns. If $\mathbf{A}$ is $m \times n$, then $\mathbf{A}^\top$ is $n \times m$:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Listing 2.1: Matrix multiplication in NumPy

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A @ B)           # matrix product -> [[19 22] [43 50]]
print(A * B)           # WATCH OUT: element-wise, NOT matrix product!

x = np.array([1, 0, -1])
W = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(W @ x)           # [-2 -2]    matrix-vector product

print(A.T)             # transpose
print(np.eye(3))       # 3x3 identity matrix
print(A.shape)         # (2, 2)    always check your shapes
```

To multiply matrices, inner dimensions must match and the result takes the outer dimensions: $(m \times n)(n \times p) \rightarrow (m \times p)$. Matrix multiplication is not commutative, and in code `@` means matrix product while `*` means element-wise — never confuse them.

Practice Problems

1. Compute $\begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 1 \end{bmatrix}$.
2. Which of these products are valid, and what shape is the result? (a) $(3 \times 5)(5 \times 2)$
(b) $(4 \times 2)(4 \times 2)$ (c) $(1 \times 7)(7 \times 1)$.
3. Write the transpose of $\begin{bmatrix} 2 & 5 \\ -1 & 0 \\ 3 & 3 \end{bmatrix}$.
4. A network layer has weight matrix $\mathbf{W}$ of shape (10×784) and input $\mathbf{x}$ of shape $(784,)$. What is the shape of $\mathbf{W}\mathbf{x}$?

Solutions.

1. $\begin{bmatrix} (2)(1) + (0)(2) & (2)(4) + (0)(1) \\ (1)(1) + (3)(2) & (1)(4) + (3)(1) \end{bmatrix} = \begin{bmatrix} 2 & 8 \\ 7 & 7 \end{bmatrix}$.
2. (a) valid $\rightarrow (3 \times 2)$; (b) invalid (inner $2 \neq 4$); (c) valid $\rightarrow (1 \times 1)$, a single number (this is just a dot product).
3. $\begin{bmatrix} 2 & -1 & 3 \\ 5 & 0 & 3 \end{bmatrix}$ (shape 2×3).
4. $(10,)$ — a 10-dimensional vector, e.g. the scores for 10 classes.

Chapter 3

Eigenvalues and Eigenvectors (Concept)

Eigenvalues sound intimidating, but the idea is visual and concrete. This chapter stays at the level of *concept and intuition* — what they are and why AI cares — rather than heavy computation.

3.1 A Matrix as a Transformation

Multiplying a vector by a matrix usually does two things at once: it *rotates* the vector (changes its direction) and *scales* it (changes its length). For example, multiplying by $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ sends most vectors off in a new direction.

But for any given matrix, a few special vectors are only *stretched or squashed* — their direction does not change at all. These are the **eigenvectors**, and the factor by which each is stretched is its **eigenvalue**.

An **eigenvector** of a matrix is a direction that the matrix does not turn — it only lengthens or shortens it. The **eigenvalue** is how much it lengthens (or shortens, or flips) along that direction.

3.2 The Definition

A nonzero vector $\mathbf{v}$ is an eigenvector of the square matrix $\mathbf{A}$ if

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

for some scalar λ , the eigenvalue. Read it aloud: “matrix times special vector equals just a *number* times that same vector.” The heavy machinery of the matrix collapses, on that special direction, into simple scaling.

Checking an eigenvector

Let $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

$$\mathbf{A}\mathbf{v} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The output points the *same* way as $\mathbf{v}$, just three times longer. So $\mathbf{v}$ is an eigenvector with eigenvalue $\lambda = 3$.

Now try $\mathbf{u} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$:

$$\mathbf{A}\mathbf{u} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \cdot \mathbf{u},$$

so $\mathbf{u}$ is another eigenvector, this time with eigenvalue $\lambda = 1$ (unchanged in length).

Picture a rubber sheet being stretched. Almost every arrow drawn on the sheet swings to a new angle as you pull. But along the two “grain” directions of the stretch, arrows stay pointing the same way and simply get longer or shorter. Those grain directions are the eigenvectors; the stretch factors are the eigenvalues. Every matrix has its own hidden grain.

3.3 Why AI Cares

Eigenvalues and eigenvectors are the mathematical heart of **Principal Component Analysis (PCA)**, the classic technique for *dimensionality reduction*. PCA finds the directions in your data along which it varies the most — these directions are the eigenvectors of the data’s covariance matrix, and the eigenvalues tell you how much variance lies along each. Keeping only the few directions with the largest eigenvalues lets you compress high-dimensional data (say, 784-pixel images) down to a handful of numbers while losing very little information.

The same ideas power spectral clustering, Google’s original PageRank algorithm, and the analysis of whether training will be stable (the eigenvalues of certain matrices tell you if gradients will explode or vanish).

3.4 A Few Facts to Carry Forward

- Only **square** matrices ($n \times n$) have eigenvalues/eigenvectors.
- An $n \times n$ matrix has at most n eigenvalues.
- Eigenvalues can be zero, negative, or even complex. A negative eigenvalue means the eigenvector is flipped to point the opposite way.
- If an eigenvalue is 0, the matrix squashes that direction flat — a sign the matrix is “singular” (has no inverse).

Listing 3.1: Eigenvalues in NumPy

```
import numpy as np

A = np.array([[2, 1],
              [1, 2]])

values, vectors = np.linalg.eig(A)
```

```

print(values)      # [3. 1.]    the eigenvalues
print(vectors)     # each COLUMN is an eigenvector (normalised to length
                    # 1)

# Verify  $A v = \lambda v$  for the first eigenpair
v = vectors[:, 0]
print(A @ v)       # equals values[0] * v
print(values[0] * v)

```

Practice Problems

1. In your own words, what does it mean for $\mathbf{v}$ to be an eigenvector of $\mathbf{A}$?
2. Verify that $\mathbf{v} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ is an eigenvector of $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$, and find its eigenvalue.
3. An eigenvalue of a matrix is -1 . Describe geometrically what the matrix does to the corresponding eigenvector.
4. In PCA, you compute eigenvalues of the covariance matrix and keep the largest ones. What are you trying to preserve by doing so?

Solutions.

1. $\mathbf{A}$ acting on $\mathbf{v}$ does not change its direction; it only scales it. Formally $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$.
2. $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix} = 5 \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, so it is an eigenvector with eigenvalue 5.
3. It flips the eigenvector to point in the exact opposite direction while keeping its length the same (scaled by -1).
4. The directions of greatest *variance* in the data — i.e. as much of the information/spread as possible, using as few dimensions as possible.

Part II

Probability and Statistics

Machine learning is, at its core, reasoning under uncertainty. A model never “knows” the answer; it assigns *probabilities* and updates them as it sees data. Statistics is how we summarise data before a model ever touches it, and how we judge whether the model is any good afterwards. These two subjects are the grammar of every prediction an AI makes.

Chapter 4

Probability Fundamentals

4.1 The Basic Vocabulary

An **experiment** is any process with an uncertain outcome (rolling a die, flipping a coin, whether an email is spam). The set of all possible outcomes is the **sample space**, written S . An **event** is any subset of outcomes we care about — for a die, “rolling an even number” is the event $\{2, 4, 6\}$.

The **probability** of an event A , written $P(A)$, is a number between 0 and 1 measuring how likely it is:

$$0 \leq P(A) \leq 1, \quad P(\text{something happens}) = 1.$$

$P(A) = 0$ means impossible, $P(A) = 1$ means certain. For equally likely outcomes,

$$P(A) = \frac{\text{number of outcomes in } A}{\text{total number of outcomes}}.$$

Worked Example

Roll one fair six-sided die. The sample space is $S = \{1, 2, 3, 4, 5, 6\}$.

$$P(\text{even}) = \frac{|\{2, 4, 6\}|}{6} = \frac{3}{6} = \frac{1}{2}, \quad P(\text{greater than 4}) = \frac{|\{5, 6\}|}{6} = \frac{2}{6} = \frac{1}{3}.$$

4.2 Combining Events

4.2.1 Complement — the “not” rule

The probability that A does *not* happen is

$$P(\text{not } A) = 1 - P(A).$$

This tiny rule is a workhorse: it is often far easier to compute the chance of the opposite event. “At least one” problems almost always yield to $P(\text{at least one}) = 1 - P(\text{none})$.

4.2.2 Addition — the “or” rule

For the probability that A *or* B happens:

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B).$$

We subtract the overlap so it is not counted twice. If the events cannot both happen (they are **mutually exclusive**), the overlap is 0 and it simplifies to $P(A) + P(B)$.

4.2.3 Multiplication — the “and” rule

For the probability that A and B both happen:

$$P(A \text{ and } B) = P(A) P(B | A),$$

where $P(B | A)$ is the probability of B *given that* A has happened (a **conditional probability**, our next topic). If A and B are **independent** — one has no effect on the other — then $P(B | A) = P(B)$ and the rule simplifies to $P(A)P(B)$.

Worked Example

Draw two cards from a standard 52-card deck *without replacement*. What is $P(\text{both aces})$? The first card is an ace with probability $4/52$. Given that, only 3 aces remain in 51 cards, so the second is an ace with probability $3/51$.

$$P(\text{both aces}) = \frac{4}{52} \times \frac{3}{51} = \frac{12}{2652} = \frac{1}{221} \approx 0.0045.$$

The events are *not* independent — removing the first ace changed the second probability.

4.3 Conditional Probability

The **conditional probability** of A given B is the probability of A once we *know* B occurred:

$$P(A | B) = \frac{P(A \text{ and } B)}{P(B)}, \quad P(B) > 0.$$

Conditioning means *shrinking your world*. Before you know anything, the whole sample space is in play. The moment you learn “ B happened,” you throw away every outcome outside B and re-scale what remains so it adds to 1. Learning is nothing more than repeatedly shrinking the world as evidence arrives.

Every classifier is a conditional-probability machine. A spam filter estimates $P(\text{spam} | \text{words in the email})$. An image model estimates $P(\text{cat} | \text{pixels})$. Training the model means tuning it so these conditional probabilities match reality as closely as possible. Independence assumptions (like “treat each word separately”) are what make the classic Naive Bayes classifier fast enough to be practical.

Listing 4.1: Simulating probability in NumPy

```
import numpy as np
rng = np.random.default_rng(0)

# Estimate P(even) on a die by simulation
rolls = rng.integers(1, 7, size=100_000)
print(np.mean(rolls % 2 == 0))    # ~0.5

# Estimate P(sum == 7) for two dice
```

```
d1 = rng.integers(1, 7, size=100_000)
d2 = rng.integers(1, 7, size=100_000)
print(np.mean(d1 + d2 == 7))      # ~0.1667 (= 6/36)
```

Probabilities live in $[0, 1]$. Use “not” ($1 - P$), “or” (add, subtract the overlap), and “and” (multiply, using a conditional if the events interact). Conditioning on evidence means shrinking the sample space to what is still possible.

Practice Problems

1. A bag has 3 red and 5 blue balls. What is $P(\text{red})$? What is $P(\text{not red})$?
2. You flip a fair coin 3 times. What is $P(\text{at least one head})$? (Hint: use the complement.)
3. $P(A) = 0.6$, $P(B) = 0.5$, $P(A \text{ and } B) = 0.3$. Find $P(A \text{ or } B)$. Are A and B independent?
4. In a class, 70% passed maths, and of those, 80% also passed physics. What is $P(\text{passed both})$?

Solutions.

1. $P(\text{red}) = 3/8 = 0.375$; $P(\text{not red}) = 1 - 3/8 = 5/8 = 0.625$.
2. $P(\text{no heads}) = (1/2)^3 = 1/8$, so $P(\text{at least one head}) = 1 - 1/8 = 7/8 = 0.875$.
3. $P(A \text{ or } B) = 0.6 + 0.5 - 0.3 = 0.8$. Independent would require $P(A \text{ and } B) = P(A)P(B) = 0.30$. It equals 0.3, so yes — they are independent.
4. $P(\text{both}) = P(\text{maths}) P(\text{physics} \mid \text{maths}) = 0.70 \times 0.80 = 0.56$.

Chapter 5

Bayes' Theorem

Bayes' theorem is a single formula for *updating a belief when new evidence arrives*. It is arguably the most important equation in all of machine learning, and it follows in two lines from the conditional-probability rule you just learned.

5.1 Deriving It

We know two ways to write the probability of “ A and B ”:

$$P(A \text{ and } B) = P(A | B) P(B) = P(B | A) P(A).$$

Setting the last two equal and dividing by $P(B)$ gives **Bayes' theorem**:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

5.2 Reading the Formula

Bayes' theorem lets you *flip* a conditional probability — to go from the one you can measure to the one you actually want. The pieces have names worth memorising:

Term	Meaning
$P(A)$	prior — belief in A <i>before</i> seeing evidence
$P(B A)$	likelihood — how well A explains the evidence B
$P(A B)$	posterior — updated belief in A <i>after</i> seeing B
$P(B)$	evidence — total probability of seeing B

Bayes says: *new belief* $\propto$ *old belief* $\times$ *how well the belief explains the new data*. Strong prior beliefs need strong evidence to overturn; weak priors are swayed easily. This is exactly how a careful person updates their mind, written as arithmetic.

5.3 The Classic Medical-Test Example

This example surprises almost everyone the first time, and it shows why Bayes' theorem matters.

The base-rate trap

A disease affects 1% of people. A test is 99% accurate: it correctly flags 99% of sick people, and correctly clears 99% of healthy people. *You test positive*. What is the probability you are actually sick?

Let D = “has disease,” $+$ = “tests positive.” We want $P(D \mid +)$.

$$P(D) = 0.01, \quad P(+ \mid D) = 0.99, \quad P(+ \mid \text{no } D) = 0.01.$$

First the evidence $P(+)$ — the total chance of a positive, from sick and healthy people combined:

$$P(+)=\underbrace{P(+ \mid D)P(D)}_{\text{true positive}}+\underbrace{P(+ \mid \text{no } D)P(\text{no } D)}_{\text{false positive}}=(0.99)(0.01)+(0.01)(0.99)=0.0198.$$

Now Bayes:

$$P(D \mid +)=\frac{P(+ \mid D)P(D)}{P(+)}=\frac{(0.99)(0.01)}{0.0198}=\frac{0.0099}{0.0198}=0.5.$$

Despite a “99% accurate” test, there is only a **50%** chance you are sick! Because the disease is so rare, the healthy majority produces just as many false positives as there are true positives among the sick.

Bayes' theorem: $\text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$. Always account for the prior (the base rate). Ignoring it is the single most common mistake people make with probability.

Bayesian reasoning underlies a whole family of methods: the **Naive Bayes** classifier (still a strong baseline for text and spam), Bayesian networks, Bayesian optimisation for tuning hyperparameters, and the entire field of Bayesian deep learning, where a model reports not just a prediction but its *uncertainty* about that prediction. Whenever you hear “the model updated its belief given the data,” Bayes is in the room.

Listing 5.1: Bayes' theorem as a function

```
def bayes(prior, likelihood, false_alarm):
    """P(D|+) given P(D), P(+|D), and P(+|not D)."""
    evidence = likelihood * prior + false_alarm * (1 - prior)
    return likelihood * prior / evidence

print(bayes(prior=0.01, likelihood=0.99, false_alarm=0.01))    # 0.5

# Test twice: feed the first posterior back in as the new prior
p1 = bayes(0.01, 0.99, 0.01)    # 0.50 after one positive
p2 = bayes(p1, 0.99, 0.01)      # ~0.99 after a second positive
print(p2)
```

Practice Problems

1. Write Bayes' theorem from memory and name all four parts.
2. A factory has two machines. Machine X makes 60% of parts with a 2% defect rate; machine Y makes 40% with a 5% defect rate. A part is defective — what is the probability it came from machine Y?
3. In the medical example, redo the calculation if the disease affects 10% of people (instead of 1%), keeping the test 99% accurate. What is $P(D | +)$ now, and why did it change so much?

Solutions.

1. $P(A | B) = \frac{P(B | A)P(A)}{P(B)}$; prior $P(A)$, likelihood $P(B | A)$, posterior $P(A | B)$, evidence $P(B)$.
2. Evidence $P(\text{def}) = (0.02)(0.6) + (0.05)(0.4) = 0.012 + 0.020 = 0.032$. $P(Y | \text{def}) = \frac{0.020}{0.032} = 0.625$.
3. $P(+) = (0.99)(0.10) + (0.01)(0.90) = 0.099 + 0.009 = 0.108$; $P(D | +) = 0.099/0.108 \approx 0.917$. A higher prior (base rate) means a positive result is much stronger evidence — the posterior jumps from 50% to about 92%.

Chapter 6

Mean, Median, and Mode

We now turn from probability (reasoning about the future) to statistics (summarising data we already have). The first job of statistics is to answer a simple question: *where is the centre of my data?* There are three standard answers.

6.1 The Three Measures of Centre

6.1.1 Mean — the average

The **mean** (arithmetic average) adds up all values and divides by the count. For n values $x_1, \dots, x_n$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{x_1 + x_2 + \dots + x_n}{n}.$$

6.1.2 Median — the middle

The **median** is the middle value when the data is sorted. With an odd count it is the single middle number; with an even count it is the average of the two middle numbers.

6.1.3 Mode — the most common

The **mode** is the value that appears most often. Data can have one mode, several modes, or no mode at all.

Worked Example

Dataset: 3, 7, 7, 2, 9, 7, 5. Sorted: 2, 3, 5, 7, 7, 7, 9 ($n = 7$).

- **Mean:** $(3 + 7 + 7 + 2 + 9 + 7 + 5)/7 = 40/7 \approx 5.71$.
- **Median:** the 4th of 7 sorted values = 7.
- **Mode:** 7 (appears three times).

6.2 Which One Should You Use?

The choice matters because the mean is sensitive to **outliers** — a single extreme value drags it around — while the median is not.

Why the median resists outliers

Five salaries (in thousands): 40, 42, 45, 48, 50. Mean = 45, median = 45 — they agree. Now the CEO joins with a salary of 1000: 40, 42, 45, 48, 50, 1000. Mean jumps to ≈ 204 , but the median is only $(45 + 48)/2 = 46.5$. The median still describes a typical worker; the mean has been hijacked by one value.

Use the **mean** for roughly symmetric data with no wild outliers (heights, test scores). Use the **median** for skewed data (incomes, house prices, waiting times), where a few huge values would distort the average. Use the **mode** for categories — the most common colour, the most frequent word, the best-selling product.

Centre statistics run quietly through machine learning. *Mean imputation* fills in missing values with the column mean (or median, when the column is skewed). Standardising features — subtracting the mean so each feature is centred at zero — is a routine preprocessing step that helps many models train faster. And the “mean squared error” loss is minimised exactly when a model predicts the mean of the targets.

Listing 6.1: Centre statistics in NumPy / SciPy

```
import numpy as np
from scipy import stats

x = np.array([3, 7, 7, 2, 9, 7, 5])

print(np.mean(x))           # 5.714...
print(np.median(x))         # 7.0
print(stats.mode(x, keepdims=False).mode) # 7

# The mean's sensitivity to an outlier
y = np.append(x, 1000)
print(np.mean(y), np.median(y)) # mean explodes; median barely moves
```

Mean = average (sensitive to outliers), median = middle (robust to outliers), mode = most frequent (works for categories). Match the measure to the shape of your data.

Practice Problems

1. Find the mean, median, and mode of 4, 8, 6, 4, 10, 4, 8.
2. House prices in a neighbourhood are mostly around \$300k, but one mansion sold for \$5M. Which centre measure best describes a “typical” house, and why?
3. A dataset has mean 50 and median 50. Then you add one value of 500. Which changes more, the mean or the median?

Solutions.

1. Sorted: 4, 4, 4, 6, 8, 8, 10. Mean = $44/7 \approx 6.29$; median = 6 (4th value); mode = 4 (three times).
2. The **median** — the \$5M mansion is an outlier that would inflate the mean far above what a normal buyer would pay.

3. The **mean** changes much more; the median moves at most to the next value up, while the mean is pulled toward 500.

Chapter 7

Variance and Standard Deviation

Knowing the centre of your data is only half the story. Two datasets can share the same mean yet look completely different — one tightly clustered, one wildly spread out. **Variance** and **standard deviation** measure that spread.

7.1 Building Up the Idea

Start with the **deviation** of each value from the mean: $x_i - \bar{x}$. Some are positive, some negative, and they always sum to zero — so we cannot just average them. We *square* them first (killing the signs), then average.

7.1.1 Variance

The **variance** is the average of the squared deviations:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Large variance means values sit far from the mean on average; small variance means they huddle close.

7.1.2 Standard deviation

Because variance is in *squared* units (squared rupees, squared centimetres), it is hard to interpret. Taking the square root returns us to the original units and gives the **standard deviation**:

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

Standard deviation is the “typical distance” of a data point from the mean. If a class has mean score 70 with standard deviation 5, most students scored roughly 65–75. If instead the standard deviation were 20, scores would be scattered from 50 to 90. Same average, very different classroom.

Worked Example

Dataset: 2, 4, 4, 4, 5, 5, 7, 9 ($n = 8$). Mean: $\bar{x} = 40/8 = 5$.

Squared deviations:

$$\begin{aligned} (2-5)^2, (4-5)^2, (4-5)^2, (4-5)^2, (5-5)^2, (5-5)^2, (7-5)^2, (9-5)^2 \\ = 9, 1, 1, 1, 0, 0, 4, 16. \end{aligned}$$

Variance: $\sigma^2 = (9 + 1 + 1 + 1 + 0 + 0 + 4 + 16)/8 = 32/8 = 4$. Standard deviation: $\sigma = \sqrt{4} = 2$.
 So values typically sit about 2 units from the mean of 5 — which matches the data.

7.2 A Note on n versus $n - 1$

When your data is a *sample* from a larger population and you want to *estimate* the population variance, statisticians divide by $n - 1$ instead of n (this is *Bessel's correction*, and it corrects a slight underestimate). Dividing by n gives the *population* variance; by $n - 1$ the *sample* variance. For large n the difference is negligible, but be aware that software makes a choice: NumPy defaults to n , pandas defaults to $n - 1$.

Variance is everywhere in machine learning. Feature *standardisation* rescales each feature to have standard deviation 1, so no single feature dominates just because it happens to be measured in bigger numbers. The famous *bias-variance tradeoff* describes how a model can fail: high bias means it is too simple (underfitting); high variance means it is too sensitive to the training data (overfitting). And *batch normalisation*, a key trick in deep learning, works by adjusting the mean and variance of the signals flowing through a network.

Listing 7.1: Spread in NumPy

```
import numpy as np

x = np.array([2, 4, 4, 4, 5, 5, 7, 9])

print(np.var(x))           # 4.0    population variance (divides by n)
print(np.std(x))          # 2.0    population standard deviation
print(np.var(x, ddof=1))  # 4.57   sample variance (divides by n-1)

# Standardise a feature to mean 0, std 1 (very common preprocessing)
z = (x - x.mean()) / x.std()
print(z.mean(), z.std())  # ~0.0   1.0
```

Variance = average squared distance from the mean; standard deviation = its square root, in the original units. Standard deviation is your everyday measure of “how spread out” data is.

Practice Problems

1. Compute the mean, variance, and standard deviation of 1, 3, 5, 7, 9 (use divisor n).
2. Dataset A is {50, 50, 50, 50}; dataset B is {20, 40, 60, 80}. Without computing, which has the larger standard deviation? Then check dataset A's standard deviation.
3. Why do we square the deviations instead of just averaging them directly?

4. A feature has mean 100 and standard deviation 15. After standardising, a value of 130 becomes what number?

Solutions.

1. Mean $= 25/5 = 5$. Squared deviations: 16, 4, 0, 4, 16, sum $= 40$. Variance $= 40/5 = 8$; std $= \sqrt{8} \approx 2.83$.
2. B has the larger spread. A's values are all identical, so its standard deviation is 0.
3. The raw deviations always sum to zero (positives cancel negatives). Squaring removes the signs so the spread does not cancel out; it also penalises large deviations more heavily.
4. $z = (130 - 100)/15 = 30/15 = 2$ — the value is 2 standard deviations above the mean.

Chapter 8

Correlation

So far we have summarised *one* variable at a time. **Correlation** measures how two variables move *together*: when one goes up, does the other tend to go up, go down, or do nothing?

8.1 From Covariance to Correlation

8.1.1 Covariance

Covariance extends the idea of variance to a pair of variables. For each data point, multiply its deviation in x by its deviation in y , then average:

$$\text{Cov}(x, y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

If x and y are usually above (or below) their means together, the products are positive and covariance is positive. If one is high when the other is low, the products are negative.

The trouble with covariance is that its size depends on the units — measure height in millimetres instead of metres and the covariance balloons, though nothing real has changed. We fix this by dividing out the scale.

8.1.2 The correlation coefficient

The **Pearson correlation coefficient** r is covariance divided by the two standard deviations, which cancels the units and pins the value between -1 and $+1$:

$$r = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}, \quad -1 \leq r \leq 1.$$

r	Meaning
$+1$	perfect positive linear relationship (a rising straight line)
$\approx +0.7$	strong positive: as x rises, y usually rises
0	no <i>linear</i> relationship
≈ -0.7	strong negative: as x rises, y usually falls
-1	perfect negative linear relationship (a falling straight line)

Correlation is a number between -1 and $+1$ that answers “do these two things rise and fall together?” Near ± 1 : tightly linked in a straight-line way. Near 0 : no straight-line link. The *sign* tells you the direction; the *magnitude* tells you the strength.

8.2 The Two Great Warnings

8.2.1 Correlation is not causation

A strong correlation does *not* mean one variable causes the other. Ice-cream sales and drowning deaths are strongly correlated — but ice cream does not cause drowning. A third variable, hot weather, drives both. Always ask what hidden factor might be lurking.

8.2.2 Correlation only sees straight lines

Pearson’s r measures *linear* association only. Two variables can be perfectly related by a curve (say $y = x^2$) and still have $r \approx 0$, because the relationship is not a straight line. A correlation of zero means “no linear trend,” not “no relationship.”

Worked Example

Hours studied (x) and exam score (y) for five students:

$$x : 1, 2, 3, 4, 5 \quad y : 52, 58, 62, 68, 75.$$

As study hours climb, scores climb steadily — a strong positive relationship. Computing gives $r \approx 0.995$, very close to $+1$: an almost perfectly straight-line increase. (This does not by itself *prove* studying causes higher scores, but combined with common sense it is suggestive.)

Correlation is a first-pass tool in every data project. Before building a model you compute a *correlation matrix* to see which features relate to the target — and to spot *multicollinearity*, where two input features are so highly correlated they carry redundant information and can destabilise a model. In recommendation systems, correlation between users’ rating patterns is one way to decide who has “similar taste.”

Listing 8.1: Correlation in NumPy

```
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([52, 58, 62, 68, 75])

print(np.cov(x, y, ddof=0))          # 2x2 covariance matrix
print(np.corrcoef(x, y))             # 2x2 correlation matrix
print(np.corrcoef(x, y)[0, 1])       # ~0.995 the single r value

# Zero linear correlation but a clear (quadratic) relationship
t = np.array([-2, -1, 0, 1, 2])
print(np.corrcoef(t, t**2)[0, 1])    # ~0.0 -- r misses the curve
```


Correlation $r \in [-1, 1]$ measures the strength and direction of a *linear* relationship. Remember the two warnings: correlation is not causation, and $r = 0$ rules out only *straight-line* relationships, not all relationships.

Practice Problems

1. If $r = -0.9$ between screen time and sleep quality, describe the relationship in plain words.
2. Give an example (not from the text) of two variables that might be correlated without one causing the other.
3. True or false: “ $r = 0$ means the two variables are unrelated.” Explain.
4. You find features “height in cm” and “height in inches” in a dataset, both describing the same people. What correlation do you expect, and why is keeping both a problem?

Solutions.

1. A strong *negative* linear relationship: more screen time is associated with noticeably worse sleep quality (and vice versa).
2. Many valid answers, e.g. number of firefighters at a scene and amount of fire damage (both driven by fire size), or shoe size and reading ability in children (both driven by age).
3. False. $r = 0$ means no *linear* relationship; a strong nonlinear (e.g. curved) relationship can still exist.
4. Almost exactly $r = +1$ (they are the same quantity rescaled). Keeping both adds no information and causes multicollinearity, which can make some models unstable or hard to interpret.

Chapter 9

The Normal Distribution

The **normal distribution** — the famous “bell curve” — is the most important distribution in statistics and machine learning. An astonishing range of real-world quantities, from human heights to measurement errors to the noise in data, follow it closely.

9.1 Shape and Parameters

A normal distribution is a smooth, symmetric bell shape, fully described by just two numbers:

- the **mean** μ — where the peak sits (the centre),
- the **standard deviation** σ — how wide the bell is.

We write $X \sim \mathcal{N}(\mu, \sigma^2)$ to say “ X is normally distributed with mean μ and variance σ^2 .” A small σ gives a tall, narrow bell; a large σ gives a short, wide one. The curve is always symmetric about μ , so for a normal distribution the mean, median, and mode all coincide.

9.2 The 68–95–99.7 Rule

The single most useful fact about the normal distribution is how its area is divided by standard deviations from the mean:

Range around the mean	Share of the data
$\mu \pm 1\sigma$	about 68%
$\mu \pm 2\sigma$	about 95%
$\mu \pm 3\sigma$	about 99.7%

Almost everything in a normal distribution lives within 3 standard deviations of the mean. Being more than 3σ out happens only about 0.3% of the time — which is why a “3-sigma event” is shorthand for something rare and surprising.

Worked Example

Adult heights are roughly normal with mean $\mu = 170$ cm and standard deviation $\sigma = 10$ cm. Then:

- About 68% of people are between 160 and 180 cm ($\mu \pm 1\sigma$).
- About 95% are between 150 and 190 cm ($\mu \pm 2\sigma$).
- About 99.7% are between 140 and 200 cm ($\mu \pm 3\sigma$).

Someone 200 cm tall is at $+3\sigma$ — taller than roughly 99.85% of people.

9.3 Standardising: the z-score

Any normal value can be converted to a **z-score**, which says how many standard deviations it lies from the mean:

$$z = \frac{x - \mu}{\sigma}.$$

A z-score of +2 means “two standard deviations above average.” This turns any normal distribution into the *standard* normal $\mathcal{N}(0, 1)$ (mean 0, standard deviation 1), so a single table or function can answer questions about all of them. You met this exact formula when standardising features in Chapter 7 — it is the same operation.

The normal distribution is baked into machine learning in many places. Model *weights* are typically *initialised* by drawing from a normal distribution. Many algorithms assume the noise in the data is Gaussian (another name for normal). The **Central Limit Theorem** — which says that averages of many independent quantities tend toward a normal distribution regardless of their original shape — is why the bell curve appears so often and why it underpins statistical testing. And detecting *anomalies* often amounts to flagging points that sit many standard deviations from the mean.

Listing 9.1: The normal distribution in NumPy / SciPy

```
import numpy as np
from scipy import stats

# Draw 10,000 samples from N(mean=170, std=10)
samples = np.random.default_rng(0).normal(170, 10, size=10_000)
print(samples.mean(), samples.std())    # ~170, ~10

# Fraction within one standard deviation -> should be ~0.68
within_1sd = np.mean(np.abs(samples - 170) <= 10)
print(within_1sd)                      # ~0.68

# z-score of a specific value
z = (190 - 170) / 10
print(z)                               # 2.0

# P(X <= 190) using the standard normal CDF
print(stats.norm.cdf(190, loc=170, scale=10))    # ~0.977
```

A normal distribution is a symmetric bell described by its mean μ (centre) and standard deviation σ (width). Remember 68–95–99.7 for ± 1 , ± 2 , ± 3 standard deviations, and use the z-score $z = (x - \mu)/\sigma$ to compare across different normal distributions.

Practice Problems

1. IQ scores are normal with $\mu = 100$, $\sigma = 15$. What percentage of people score between 85 and 115? Between 70 and 130?

2. Compute the z-score of an IQ of 130. How rare is scoring 130 or above, roughly?
3. For a normal distribution, why are the mean, median, and mode equal?
4. Exam scores are normal with $\mu = 65$, $\sigma = 8$. A student scores 81. How many standard deviations above the mean is that, and is it common?

Solutions.

1. 85 to 115 is $\mu \pm 1\sigma \rightarrow$ about 68%. 70 to 130 is $\mu \pm 2\sigma \rightarrow$ about 95%.
2. $z = (130 - 100)/15 = 2.0$. About 95% lie within $\pm 2\sigma$, leaving 5% in the two tails, so roughly 2.5% score 130 or above.
3. Because the distribution is perfectly symmetric about its centre, the balance point (mean), the middle value (median), and the peak (mode) all land at the same place, μ .
4. $z = (81 - 65)/8 = 2.0$, i.e. 2 standard deviations above the mean — in the top $\sim 2.5\%$, so quite uncommon.

Part III

Calculus

Calculus is the mathematics of *change*, and one idea from it powers nearly all of modern AI: the **gradient**. Every time a neural network learns, it is following gradients downhill. You do not need to master all of calculus — you need to understand slopes, derivatives, and gradients well enough to know what “the model is minimising the loss” really means.

Chapter 10

Introduction to Calculus and Gradients

10.1 Slope: the Rate of Change

Calculus begins with a simple question: *how fast is something changing?* For a straight line, the answer is the familiar slope — rise over run:

$$\text{slope} = \frac{\text{change in } y}{\text{change in } x} = \frac{\Delta y}{\Delta x}.$$

A slope of 2 means y rises 2 units for every 1 unit x moves right. A negative slope means the line goes down as you move right; a slope of 0 is flat.

But most interesting functions are curves, not straight lines, and their steepness changes from point to point. The **derivative** is the tool that gives the slope of a curve *at a single point*.

10.2 The Derivative

The **derivative** of a function $f(x)$, written $f'(x)$ or $\frac{df}{dx}$, is the slope of the curve at the point x — equivalently, the slope of the straight line that just touches the curve there (the tangent line).

Imagine zooming in on a curve. The more you magnify, the straighter any smooth curve looks, until a tiny piece is indistinguishable from a straight line. The slope of that tiny straight piece is the derivative. It tells you which way the curve is heading, and how steeply, right at that spot.

10.2.1 A few rules you will actually use

You rarely compute derivatives by hand in AI — libraries do it automatically — but recognising the pattern helps. The one rule worth memorising is the **power rule**:

$$\text{if } f(x) = x^n, \quad \text{then } f'(x) = n x^{n-1}.$$

A few consequences:

$$\frac{d}{dx}[x^2] = 2x, \quad \frac{d}{dx}[x^3] = 3x^2, \quad \frac{d}{dx}[c] = 0, \quad \frac{d}{dx}[5x] = 5.$$

(The derivative of a constant is 0 — a flat line has no slope — and a constant multiplier just carries along.)

Worked Example

Let $f(x) = x^2$. Its derivative is $f'(x) = 2x$.

- At $x = 3$: slope $= 2(3) = 6$ (steep, going up).
- At $x = 0$: slope $= 2(0) = 0$ (flat — the bottom of the parabola).
- At $x = -3$: slope $= 2(-3) = -6$ (steep, going down).

Notice the slope is 0 exactly at the minimum of the curve. That is the key fact optimisation is built on: *minima and maxima occur where the derivative is zero*.

10.3 The Gradient: Slope in Many Directions

Real models depend on not one variable but millions (every weight in the network). When a function has several inputs, its “slope” becomes a *vector* of slopes — one for each input direction. This vector is the **gradient**, written ∇f (“nabla f ” or “grad f ”).

Each entry of the gradient is a **partial derivative**: the slope in one input’s direction while all other inputs are held fixed. For a function $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}.$$

Worked Example

Let $f(x, y) = x^2 + y^2$. Holding y fixed, the slope in the x direction is $2x$; holding x fixed, the slope in the y direction is $2y$. So

$$\nabla f = \begin{bmatrix} 2x \\ 2y \end{bmatrix}.$$

At the point $(3, 1)$ the gradient is $\begin{bmatrix} 6 \\ 2 \end{bmatrix}$.

The **gradient** is a vector pointing in the direction of *steepest increase* of a function, and its length says how steep that climb is. At a minimum or maximum, the gradient is the zero vector.

Stand on a hillside in fog. You cannot see the whole landscape, but you can feel the ground under your feet. The gradient is the direction of *steepest uphill* right where you stand. To get *downhill* as fast as possible — toward the valley floor — you walk in the *opposite* direction of the gradient. That single idea is how neural networks learn.

10.4 Gradient Descent: How Models Learn

Training a model means finding the settings (weights) that make its error — the **loss** — as small as possible. The loss is a function of the weights, and we want its minimum. Since minima sit where the slope is zero, and the gradient points uphill, we repeatedly step *downhill*, against the gradient. This algorithm is **gradient descent**:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \nabla L(\mathbf{w}_{\text{old}}).$$

Here $\mathbf{w}$ is the vector of weights, L is the loss, ∇L is its gradient, and η (“eta”) is the **learning rate** — the size of each step.

- Compute the gradient of the loss (which way is uphill).
- Take a small step in the *opposite* direction (downhill).
- Repeat thousands of times until the gradient is nearly zero — you have reached a valley (a minimum of the loss).

Gradient descent is *the* learning algorithm behind essentially all modern deep learning. Training GPT-style models, image recognisers, and recommendation engines all come down to computing gradients of a loss with respect to millions or billions of weights, then nudging those weights downhill. The technique that computes all those gradients efficiently is called **backpropagation**, and it is just the chain rule of calculus applied cleverly across a network. The learning rate η is one of the most important knobs you will tune: too large and the model overshoots and diverges; too small and training crawls.

One step of gradient descent by hand

Minimise $L(w) = w^2$ (a simple bowl, minimum at $w = 0$). The gradient is $L'(w) = 2w$. Start at $w = 5$ with learning rate $\eta = 0.1$:

$$\text{Step 1: } w = 5 - 0.1(2 \cdot 5) = 5 - 1.0 = 4.0$$

$$\text{Step 2: } w = 4 - 0.1(2 \cdot 4) = 4 - 0.8 = 3.2$$

$$\text{Step 3: } w = 3.2 - 0.1(2 \cdot 3.2) = 3.2 - 0.64 = 2.56$$

Each step moves w closer to 0, the true minimum. Keep going and it converges.

Listing 10.1: Gradient descent from scratch

```
import numpy as np

# Minimise L(w) = w**2, whose gradient is 2*w
def grad(w):
    return 2 * w

w = 5.0          # starting point
eta = 0.1        # learning rate

for step in range(20):
    w = w - eta * grad(w)    # the gradient descent update
```

```
print(f"step {step+1:2d}: w = {w:.4f}")

# w marches toward 0.0, the minimum of w**2
```

Models learn by **gradient descent**: repeatedly step the weights a little way *against* the gradient of the loss. The gradient says which way is uphill; the learning rate says how big a step to take. Everything else in deep-learning training is a refinement of this loop.

Practice Problems

1. Find the derivative of $f(x) = x^3$, and evaluate it at $x = 2$.
2. For $f(x) = x^2$, at which value of x is the slope zero? What is special about that point on the curve?
3. Compute the gradient of $f(x, y) = 3x^2 + y^2$ at the point $(1, 4)$.
4. Run gradient descent by hand on $L(w) = w^2$ starting at $w = 10$ with $\eta = 0.2$. Compute the first two steps.
5. In gradient descent, why do we *subtract* the gradient rather than add it?

Solutions.

1. $f'(x) = 3x^2$; at $x = 2$, $f'(2) = 3(4) = 12$.
2. At $x = 0$. It is the minimum of the parabola — the slope is flat at the bottom of the bowl.
3. $\frac{\partial f}{\partial x} = 6x$, $\frac{\partial f}{\partial y} = 2y$, so $\nabla f = \begin{bmatrix} 6x \\ 2y \end{bmatrix}$. At $(1, 4)$: $\begin{bmatrix} 6 \\ 8 \end{bmatrix}$.
4. Gradient is $2w$. Step 1: $w = 10 - 0.2(20) = 10 - 4 = 6$. Step 2: $w = 6 - 0.2(12) = 6 - 2.4 = 3.6$.
5. The gradient points *uphill* (toward larger loss). We want to *decrease* the loss, so we move in the opposite direction — downhill — which means subtracting the gradient.

Part IV

Appendices

Chapter A

NumPy Quick Reference

Everything in this book maps directly to a line of NumPy. Keep this page beside you while you code.

A.1 Creating arrays

```
import numpy as np

np.array([1, 2, 3])           # vector
np.array([[1, 2], [3, 4]])   # matrix
np.zeros((2, 3))             # 2x3 of zeros
np.ones((3,))                # vector of ones
np.eye(3)                    # 3x3 identity matrix
np.arange(0, 10, 2)          # [0 2 4 6 8]
np.random.default_rng(0).normal(0, 1, size=(2, 2)) # random normal
```

A.2 Shapes and axes

```
A.shape           # e.g. (2, 3) -- ALWAYS check this when debugging
A.ndim            # number of axes (rank): 0 scalar, 1 vector, 2 matrix
...
A.reshape(3, 2)   # rearrange into a new shape (same data)
A.T               # transpose
```

A.3 Linear algebra

```
a @ b             # dot product / matrix multiply (NOT *)
a * b             # element-wise multiply
np.dot(a, b)       # same as a @ b
np.linalg.norm(v)  # length of a vector
np.linalg.eig(A)   # eigenvalues, eigenvectors
np.linalg.inv(A)   # matrix inverse
```

A.4 Statistics

```
np.mean(x)         # mean
np.median(x)        # median
np.var(x)           # variance (population, divides by n)
np.var(x, ddof=1)   # sample variance (divides by n-1)
np.std(x)           # standard deviation
```

```
np.corrcoef(x, y)          # correlation matrix  
np.cov(x, y)              # covariance matrix  
(x - x.mean()) / x.std()  # standardise to mean 0, std 1
```

Chapter B

Glossary

Scalar

A single number; a rank-0 tensor.

Vector

An ordered list of numbers; a rank-1 tensor. Both a list and an arrow with direction and length.

Matrix

A 2-D grid of numbers (m rows $\times$ n columns); a rank-2 tensor.

Tensor

An array with any number of axes; the rank is the number of axes.

Dot product

Multiply matching components of two vectors and sum; measures how much two vectors “agree.”

Norm

The length of a vector, $\sqrt{v_1^2 + \dots + v_n^2}$.

Matrix multiplication

Row-by-column dot products; valid only when inner dimensions match: $(m \times n)(n \times p) \rightarrow (m \times p)$.

Transpose

Flipping a matrix over its diagonal, swapping rows and columns.

Identity matrix

The matrix with 1s on the diagonal, 0s elsewhere; the “1” of matrix multiplication.

Eigenvector

A direction a matrix does not rotate, only scales.

Eigenvalue

The factor by which an eigenvector is scaled.

Probability

A number in $[0, 1]$ measuring how likely an event is.

Conditional probability

$P(A \mid B)$, the probability of A given that B occurred.

Independence

Two events where one has no effect on the other’s probability.

Bayes’ theorem

$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$; updates a prior belief into a posterior using evidence.

Prior / Posterior

Belief before / after seeing evidence.

Mean

The arithmetic average.

Median

The middle value of sorted data; robust to outliers.

Mode

The most frequently occurring value.

Variance

The average squared distance from the mean.

Standard deviation

The square root of the variance; typical distance from the mean, in original units.

Covariance

How two variables vary together (unit-dependent).

Correlation

Covariance rescaled to $[-1, 1]$; strength and direction of a *linear* relationship.

Normal distribution

The symmetric bell curve, described by mean μ and standard deviation σ ; obeys the 68–95–99.7 rule.

z-score

$z = (x - \mu)/\sigma$; how many standard deviations a value is from the mean.

Derivative

The slope of a function at a point; its instantaneous rate of change.

Gradient

The vector of partial derivatives; points in the direction of steepest increase.

Gradient descent

Iteratively stepping weights against the gradient of the loss to minimise it; the core learning algorithm of deep learning.

Learning rate

The step size η in gradient descent.

Chapter C

Where to Go Next

You now have the mathematical vocabulary that the rest of machine learning is written in. A sensible next path:

1. **Consolidate with code.** Re-implement each chapter's ideas in a Jupyter notebook — dot products, matrix multiply, Bayes, gradient descent — typing and running section by section. Building the muscle memory matters more than re-reading.
2. **Linear algebra, deeper.** Matrix inverses, solving linear systems, and the geometry of transformations. 3Blue1Brown's "Essence of Linear Algebra" pairs beautifully with this book's visual approach.
3. **Probability, deeper.** Random variables, expectation, and common distributions beyond the normal (Bernoulli, binomial, Poisson).
4. **Calculus, deeper.** The chain rule — the engine of backpropagation — and multivariable optimisation.
5. **Then: classical ML.** With this foundation, linear regression, logistic regression, and a from-scratch neural network will read as applied versions of exactly what you have learned here.

*Every large model is built from these small pieces. You now know the pieces.
Read a little, compute a lot.*